

1. Logika cyfrowa

| Układ              | Formuła / sygnatura                                                 | Opis                                                                                                                  |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Sumatory           |                                                                     |                                                                                                                       |
| Półsumator         | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                             |
| Pełny sumator (FA) | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                 |
| Sumator n-bitowy   | $n \times \text{FA}$                                                | Kaskada (ripple-carry): wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to <b>Carry Out</b> całości. |

|                     |                         |                                                                                                                          |
|---------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Układy kombinacyjne |                         |                                                                                                                          |
| Dekoder             | $n \rightarrow 2^n$     | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest <b>tylko</b> bit nr $k$ .                                           |
| Multiplekser        | $2^n + n \rightarrow 1$ | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjście przechodzi $S$ -ty bit danych.                               |
| ALU                 | $A, B, f \rightarrow C$ | Dekoder na bitach $f$ wybiera operację (np. $A + B$ , $A \mid B$ , $A \& B$ ); multipleksowanie wyników bramkami AND/OR. |

|                             |                                  |                                                                                                                                          |
|-----------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Układy sekwencyjne (pamięć) |                                  |                                                                                                                                          |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawi $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Dwa sprzężone <b>NOR</b> -y.         |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1       | Wejścia <b>S</b> / <b>R</b> bramkowane AND-em z zegarem. Stan zmienia się tylko przy <b>CLK=1</b> .                                      |
| Sterowany zboczem           | zbocze 1 → 0                     | Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w <b>momencie</b> zmiany zegara. |

2. Typy danych

| Typ    | char | short | int | long | float | double | void* |
|--------|------|-------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4   | 8    | 4     | 8      | 8     |

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

**Skróty:** **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).  
Stąd **B2U** = bity → unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

| Stała | Bity          | Wartość                          |
|-------|---------------|----------------------------------|
| UMax  | <b>11...1</b> | $2^w - 1$                        |
| TMax  | <b>01...1</b> | $2^{w-1} - 1$ ( <b>INT_MAX</b> ) |
| TMin  | <b>10...0</b> | $-2^{w-1}$ ( <b>INT_MIN</b> )    |
| −1    | <b>11...1</b> | te same bity co UMax             |

**Promocja w C:** **unsigned** i **int** w jednym wyrażeniu/porównaniu ( $< > ==$ ) → **int** promowany do **unsigned** (bity bez zmian,  $-1 \rightarrow \text{UMax}$ ).

4. Rozszerzanie, obcinanie i przesunięcia

|                                |                                                |
|--------------------------------|------------------------------------------------|
| Rozszerz. 0 ( <b>zext</b> )    | <b>unsigned</b> : dopisuje 0 z góry            |
| Rozszerz. znak ( <b>sext</b> ) | <b>signed</b> : powiela bit znaku (MSB)        |
| $x \ll k$                      | $x \cdot 2^k$ (sgn./uns.)                      |
| $u \gg k$                      | $\lfloor u/2^k \rfloor$ - logiczne (zera)      |
| $x \gg k$                      | arytmetyczne (kopia MSB), zaokr. do $-\infty$  |
| $x/2^k$ do 0                   | $(x + (1 \ll k) - 1) \gg k$                    |
| Strength reduction             | <b>x*24</b> → <b>(x&lt;&lt;5)-(x&lt;&lt;3)</b> |

5. Operacje, overflow i endianness

|                                       |                                               |          |
|---------------------------------------|-----------------------------------------------|----------|
| UAdd/TAdd                             | $(u + v) \bmod 2^w$                           |          |
| Nadmiar (+)                           | $u, v > 0$ , a wynik $< 0$ (wynik $-2^w$ )    |          |
| Nadmiar (-)                           | $u, v < 0$ , a wynik $\geq 0$ (wynik $+2^w$ ) |          |
| Negacja U2                            | <code>-x = ~x+1 ; -TMin=TMin , -0=0</code>    |          |
| Wykr. nadmiaru ( <code>s=x+y</code> ) | <code>((s^x)&amp;(s^y))&gt;&gt;(w-1)</code>   |          |
| Maska / abs                           | <code>m=x&gt;&gt;31; abs=(x^m)-m</code>       |          |
| Schemat                               | Najniższy adres                               | 0x0123   |
| Big Endian                            | MSB                                           | [01][23] |
| Little Endian                         | LSB                                           | [23][01] |

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

| Format | bity | s | exp (k) | frac (n) |
|--------|------|---|---------|----------|
| float  | 32   | 1 | 8       | 23       |
| double | 64   | 1 | 11      | 52       |

Kategorie wartości

| Kategoria      | exp                                  | Własności                                                                                                                                                             |
|----------------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $\neq 0 \dots 0$<br>$\neq 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najgęściej ułożone wokół 0.                                                                                 |
| Zdenormaliz.   | $= 0 \dots 0$                        | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują $\pm 0.0$ (frac = 0) i liczby<br>bardzo bliskie zera.                                                   |
| Specjalne      | $= 1 \dots 1$                        | $\text{frac} = 0 \rightarrow \pm \infty$ (nadmiar, dzielenie<br>przez 0).<br>$\text{frac} \neq 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$ , $\infty -$<br>$\infty$ ). |

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



**G** - ostatni zachowany, **R** - pierwszy usunięty, **S** - OR reszty

| Warunek         | Ułamek | Akcja                                                             |
|-----------------|--------|-------------------------------------------------------------------|
| <b>R=0</b>      | < 0.5  | W dół (odcięcie)                                                  |
| <b>R=1, S=1</b> | > 0.5  | W górę (+1 do <b>G</b> )                                          |
| <b>R=1, S=0</b> | = 0.5  | <b>Do parzystej:</b> w górę gdy <b>G=1</b> , w dół gdy <b>G=0</b> |

Własności matematyczne i rzutowanie

|                                              |                                                                                    |
|----------------------------------------------|------------------------------------------------------------------------------------|
| Brak łączności                               | $(a + b) + c \neq a + (b + c)$ - zaokrąglenia                                      |
| Brak rozdzielności                           | $a(b + c) \neq ab + ac$                                                            |
| <b>int</b> $\rightarrow$ <b>float</b>        | możliwa utrata precyzji (zaokrągła)                                                |
| <b>int</b> $\rightarrow$ <b>double</b>       | bezstratne (52 > 32 bity)                                                          |
| <b>float/double</b> $\rightarrow$ <b>int</b> | obcina do 0; poza zakresem lub <b>NaN</b> $\rightarrow$ <b>TMin</b> ( 0x80000000 ) |

7. Rejestry x86\_64

|             |                                 |             |                                 |
|-------------|---------------------------------|-------------|---------------------------------|
| <b>%rax</b> | <b>%eax</b><br><b>%ah %al</b>   | <b>%rbx</b> | <b>%ebx</b><br><b>%bh %bl</b>   |
| <b>%rcx</b> | <b>%ecx</b><br><b>%ch %cl</b>   | <b>%rdx</b> | <b>%edx</b><br><b>%dh %dl</b>   |
| <b>%rsi</b> | <b>%esi</b><br><b>%si %sil</b>  | <b>%rdi</b> | <b>%edi</b><br><b>%di %dil</b>  |
| <b>%rbp</b> | <b>%ebp</b><br><b>%bp %bpl</b>  | <b>%rsp</b> | <b>%esp</b><br><b>%spl</b>      |
| <b>%r8</b>  | <b>%r8d</b><br><b>%r8w %r8b</b> | <b>%r9</b>  | <b>%r9d</b><br><b>%r9w %r9b</b> |

**Uwaga:** Rejestry od %r10 do %r15 używają schematu:  
**%r10**, **%r10d**, **%r10w**, **%r10b**.

Podział ról rejestrów

|                                         |                                                                                |
|-----------------------------------------|--------------------------------------------------------------------------------|
| <b>%rax</b>                             | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| <b>%rdi, %rsi, %rdx, %rcx, %r8, %r9</b> | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| <b>%rsp</b>                             | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| <b>%rbp</b>                             | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| <b>%rbx, %r12-%r15</b>                  | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| <b>%rip</b>                             | Wskaźnik instrukcji (Program Counter).                                         |

Rejestry wektorowe (zmiennoprzecinkowe)

|                     |                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>%xmm0-%xmm15</b> | 128-bitowe. <b>%xmm0</b> to wartość zwracana ( <b>float</b> / <b>double</b> ). Pierwsze 8 to argumenty. Caller-saved. |
| <b>%ymm0-%ymm15</b> | 256-bitowe rozszerzenie XMM. Dzielą z nimi najmłodsze 128 bitów.                                                      |

8. Tryby adresowania (operandy)

| Tryb                   | Format                 | Obliczanie adresu     |
|------------------------|------------------------|-----------------------|
| Natychnmiastowy (Imm)  | <b>\$Imm</b>           | <b>Imm</b>            |
| Rejestrowy (Reg)       | <b>%Ra</b>             | <b>%Ra</b>            |
| Bezpośredni (Mem)      | <b>Imm</b>             | <b>M[Imm]</b>         |
| Pośredni (Mem)         | <b>C(%Rb)</b>          | <b>M[%Rb]</b>         |
| Z przesunięciem        | <b>D(C%Rb)</b>         | <b>M[%Rb + D]</b>     |
| Skalowany (Ind/scaled) | <b>D(C%Rb, %Ri, S)</b> | <b>M[%Rb+%Ri*S+D]</b> |
| Skalowany (baseless)   | <b>(, %Ri, S)</b>      | <b>M[%Ri * S]</b>     |

**Legenda:** **Imm** / **D** to stała liczbowa, **%Ra** / **%Rb** to rejestr bazowy, **%Ri** to rejestr indeksowy, **a** / **S** to skala (tylko: 1, 2, 4 lub 8).

## 9. Flagi stanu i sterowanie

|    |                                                               |
|----|---------------------------------------------------------------|
| ZF | Zero. Wynik to 0 (np. argumenty są równe).                    |
| SF | Sign. Wynik jest ujemny (MSB = 1).                            |
| CF | Carry. Przepełnienie dla liczb <b>bez znaku</b> (unsigned).   |
| OF | Overflow. Przepełnienie dla liczb <b>ze znakiem</b> (signed). |

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

| Sufiks               | Znaczenie                               |
|----------------------|-----------------------------------------|
| <code>e / z</code>   | Equal / Zero (równe / wynik to 0)       |
| <code>ne / nz</code> | Not Equal / Not Zero (nierówne)         |
| <code>s</code>       | Sign (wynik ujemny, <code>SF=1</code> ) |

### Liczyby ze znakiem (signed)

|                     |                            |
|---------------------|----------------------------|
| <code>g / ge</code> | Greater / Greater or Equal |
| <code>l / le</code> | Less / Less or Equal       |

### Liczyby bez znaku (unsigned)

|                     |                                   |
|---------------------|-----------------------------------|
| <code>a / ae</code> | Above / Above or Equal (No Carry) |
| <code>b / be</code> | Below / Below or Equal (Carry)    |

Translacja struktur kontrolnych (C → ASM)

- if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyłuskaniach (`*p`) i efektach ubocznych (`x++`).
- switch:** tablica skoków → czas  $O(1)$ . Skok pośredni: `jmp *.L4(,%rdi,8)`. Brak `break` ⇒ **fall-through**.
- Pętla for:** zawsze redukowana do **while**:  
`init; while(cond) { body; update; }`

Wzorce assemblerowe dla pętli:

| do-while                                  | while (Jump-to-mid)                                       | while (Guarded)                                                  |
|-------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| <pre>loop:   Body   if(T) goto loop</pre> | <pre>goto test loop:   Body test:   if(T) goto loop</pre> | <pre>if(!T) goto done loop:   Body   if(T) goto loop done:</pre> |

## 10. Procedury i stos

Stos rośnie w dół (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest`: odkłada adres powrotu na stos i skacze do `dest`.
- `ret`: zdejmuje adres powrotu ze stosu i skacze pod niego.

## 11. Konwencja Wywoływań (System V ABI)

`%rdi` → `%rsi` → `%rdx` → `%rcx` → `%r8` → `%r9`

Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

Rejestry caller-saved vs callee-saved

### Caller-saved

(Wołający musi zapisać)  
Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed `call`.

### Callee-saved

(Wołany musi przywrócić)  
Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed `ret`.

`%rbx`, `%rbp`, `%r12` - `%r15`

`%rax`, `%rdi`, `%rsi`,  
`%rdx`, `%rcx`, `%r8` - `%r11`

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd **rekurencja** działa naturalnie).

**Ramka potrzebna, gdy:** brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (`&`).

### Prolog

```
pushq %rbp      ;
zapisz stary base ptr
movq %rsp, %rbp ;
nowy base ptr = rsp
subq $32, %rsp  ;
alokacja 32 bajtów
```

### Epilog

```
addq $32, %rsp ;
zwolnij alokację
popq %rbp      ;
przywróć base ptr
ret            ;
skok powrotny
```

`leave`: zastępuje `movq %rbp, %rsp` + `popq %rbp` jedną instrukcją.

## 12. Architektura CPU

Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

Pipelining i hazardy

Nakładanie faz zwiększa throughput, ale rodzi konflikty (hazardy):

### Danych

Odczyt po zapisie - wynik jeszcze nie jest w rejestrze, a kolejna instrukcja już go potrzebuje.  
**Rozwiązanie:** forwarding/bypassing (wynik z ALU prosto na wejście) lub stall/bubble.

### Sterowania

Skok wymusza odgadnięcie następnego adresu.  
**Rozwiązanie:** branch prediction. Pomyłka → czyszczenie potoku (misprediction penalty).

Out-of-Order

Nowoczesne procesory nie wykonują instrukcji sekwencyjnie:

- Superskalarność:** wiele instrukcji na cykl (wiele jednostek wykonawczych).
- Register renaming:** mapowanie rejestrów logicznych (`%rax`) na wiele fizycznych - eliminuje fałszywe zależności.
- Reorder buffer:** instrukcje liczone asynchronicznie, ale zatwierdzane w kolejności programu (spójność).

## 13. Tablice i struktury

Reprezentacja tablic w pamięci

| Typ            | Deklaracja             | Adres                                                                        |
|----------------|------------------------|------------------------------------------------------------------------------|
| Jednowymiarowa | <code>T A[1]</code>    | $A[i] = x_A + i \times \text{sizeof}(T)$                                     |
| Wielowymiarowa | <code>T A[R][C]</code> | $A[i][j] = x_A + (i \times C + j) \times \text{sizeof}(T)$                   |
| Wielopoziomowa | <code>T *A[1]</code>   | $M[x_A + i \times 8] + j \times \text{sizeof}(T)$<br>(dwa odczyty z pamięci) |

Wyrównanie danych i padding

- Zasada ogólna:** obiekt rozmiaru  $K$  leży pod adresem podzielnym przez  $K$ .
- Struktury:** każde pole wyrównane do własnego  $K$ ; rozmiar całości podzielny przez największe wyrównanie (padding na końcu).
- Optymalizacja:** pola od największego do najmniejszego → minimalny padding.

## 14. Układ pamięci

|                                                    |                                                                                |
|----------------------------------------------------|--------------------------------------------------------------------------------|
| Mapa pamięci procesu                               |                                                                                |
| Stack ↓                                            | Rośnie w dół. Zmienne lokalne, adresy powrotu (limit 8MB).                     |
| Shared libs                                        | Współdzielone biblioteki (np. <code>libc.so</code> ).                          |
| Sztywna ↑                                          | Rośnie w górę. Dynamiczna alokacja ( <code>malloc</code> , <code>new</code> ). |
| Data                                               | Zmienne globalne i statyczne (zainicjowane i niezainicjowane).                 |
| Text                                               | Read-only. Binarny kod wykonywalny programu.                                   |
| Góra tabeli = wysokie adresy, dół = niskie adresy. |                                                                                |

|                                 |                                                                                                                                                          |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Zagrożenia i mechanizmy obronne |                                                                                                                                                          |
| Buffer overflow                 | Brak sprawdzania granic tablic. Nadpisuje stos (stary <code>zrbp</code> , adres powrotu) → przejęcie sterowania.                                         |
| ROP (Gadżety)                   | Return-Oriented Programming. Sklejanie legalnych skrawków kodu zakończonych <code>ret</code> - omija zakaz wykonywania (NX).                             |
| Stack canaries                  | Losowa wartość tuż przed adresem powrotu. Przed <code>ret</code> weryfikacja ( <code>xor</code> + <code>je</code> ); uszkodzona → <code>abort()</code> . |
| NX / ASLR                       | <b>NX</b> : zakaz wykonywania kodu ze stosu i sterty.<br><b>ASLR</b> : losowanie bazowych adresów obszarów przy każdym starcie.                          |

Unie

Wszystkie pola współdzielą **ten sam adres** (offset 0); rozmiar = **największe pole**. Służą do manipulacji bitowych z ominięciem systemu typów (np. bity `float` jako `int`).

## 15. Optymalizacje i ich ograniczenia

Kompilator nie zoptymalizuje kodu, jeśli mogłoby to zmienić zachowanie programu (choćby dla specyficznych argumentów).

|                         |                                                                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optymalizacyjne blokady |                                                                                                                                                             |
| Aliasing pamięci        | <code>*p</code> i <code>*q</code> mogą wskazywać na to samo → wymuszony odczyt/zapis do RAM zamiast rejestru. Pomaga słowo kluczowe <code>restrict</code> . |
| Wywołania funkcji       | Funkcja w pętli może mieć ukryte efekty uboczne - nie zostanie wyrzucona poza pętlę. <b>Rozwiązanie</b> : inlining lub makra.                               |
| Arytmetyka float        | Brak łączności: $(a + b) + c \neq a + (b + c)$ . Kompilator <b>nigdy</b> nie zmieni kolejności działań (troska o precyzję).                                 |

|                                   |                                                                                                                           |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Podstawowe techniki optymalizacji |                                                                                                                           |
| Code motion                       | Wyrzucenie niezmienników (np. <code>x * y</code> ) poza pętlę.                                                            |
| Strength reduction                | Kosztowne → tańsze (np. <code>x * 64</code> → <code>x &lt;&lt; 6</code> , iteracja wskaźnikiem zamiast mnożenia indeksu). |
| Share subexpr.                    | Jednokrotne obliczanie powtarzających się podwyrażeń.                                                                     |

Loop unrolling      Rozwinięcie pętli (np. krok co 2). Mniejszy narzut pętli, więcej równoległości dla CPU.

## 16. Linkowanie i konsolidacja

Fazy budowania programu

```
cpp (Preprocessor) → cc1 (Kompilator) → as (Assembler) → ld (Linker)
```

Formaty plików obiektowych (ELF)

- Relokowalne ( .o )**: kod i dane gotowe do połączenia z innymi `.o`.
- Wykonywalne ( a.out )**: sklejone, gotowe do załadowania i uruchomienia.
- Współdzielone ( .so )**: linkowane dynamicznie przy ładowaniu lub w trakcie działania.

|                   |                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| .text             | Skompilowany kod maszynowy.                                                                            |
| .rodata           | Dane tylko do odczytu (np. <code>"Witaj"</code> w <code>printf</code> , tablice <code>switch</code> ). |
| .data             | Zainicjowane zmienne globalne i statyczne (np. <code>int x = 5;</code> ).                              |
| .bss              | Niezainicjowane globalne/statyczne - nie zajmują miejsca w pliku.                                      |
| .symtab           | Tablica symboli (funkcje i zmienne globalne).                                                          |
| .rel.text / .data | Wpisy relokacji: gdzie i jak linker ma wstawić ostateczne adresy.                                      |

Symbol resolution

Linker rozróżnia 3 typy symboli:

- Globalne**: zdefiniowane tu, używane tam,
- Zewnętrzne**: `extern` - używane tu, zdefiniowane gdzie indziej,
- Lokalne**: z C-owym słowem `static`.

Uwaga: Zmienne lokalne na stosie **NIE** są w ogóle widoczne dla linkera

**Silne (Strong)**: Funkcje i zainicjowane zmienne globalne (np. `int foo = 5;`).

**Słabe (Weak)**: Niezainicjowane zmienne globalne (np. `int foo;`).

- Reguła 1**   Wiele silnych → **błąd linkowania** (Multiple definition).
- Reguła 2**   1 silny + *n* słabych → wygrywa **silny**.
- Reguła 3**   Wiele słabych → linker wybiera **dowolny** (`gcc -fno-common` zakazuje).

Relokacja

Linker skleja sekcje `.text` / `.data` z plików `.o` w jeden blok, nadaje finalne adresy (run-time) i poprawia wszystkie odniesienia w kodzie wg wpisów z `.rel`.

|                                                                                                                                                                                                                                                  |                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Biblioteki</b>                                                                                                                                                                                                                                |                                                                                                                                                                         |
| <b>Statyczne ( .a )</b><br>Zbiór plików <code>.o</code> . Linker kopiuje <b>tylko używane moduły</b> .<br>Wady: każda aplikacja ma własną kopię w RAM, zmiana wymusza relink.<br><b>Flagi gcc</b> : <code>.a</code> podaje się na końcu komendy. | <b>Dynamiczne ( .so )</b><br>Kod ładowany do pamięci raz; adresy dostarczane w trakcie działania (przez GOT).<br>Zalety: znacznie mniejsze binarki, łatwe aktualizacje. |

## 17. Katalog instrukcji (AT&T)

| Opcode                      | Efekt                         | Opis                                                   |
|-----------------------------|-------------------------------|--------------------------------------------------------|
| <code>mov S, D</code>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                              |
| <code>lea S, D</code>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                |
| <code>add S, D</code>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                             |
| <code>sub S, D</code>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                           |
| <code>imul S, D</code>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                             |
| <code>sal / shl k, D</code> | $D \ll = k$                   | Przesunięcie w lewo (mnożenie przez $2^k$ ).           |
| <code>sar k, D</code>       | $D \gg = k$                   | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak.</b> |
| <code>shr k, D</code>       | $D \gg = k$                   | Logiczne w prawo. Dopełnia zerami.                     |
| <code>and / or S, D</code>  | $D \leftarrow D \& /   S$     | Operacje bitowe (ustawiają flagi).                     |
| <code>xor S, D</code>       | $D \leftarrow D \wedge S$     | Często jako <code>xor %rax, %rax</code> do zerowania.  |

### Porównania i sterowanie

|                          |                                             |                                                     |
|--------------------------|---------------------------------------------|-----------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                   | Ustawia flagi jak odejmowanie, nie zapisuje wyniku. |
| <code>test S1, S2</code> | $S2 \& S1$                                  | Ustawia flagi jak <b>AND</b> .                      |
| <code>jX dest</code>     | $\text{if}(X) \%rip \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                       |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$               | Ustawia najmłodszy bajt na 0 lub 1 wg flag.         |
| <code>cmove S, D</code>  | $\text{if}(X) D \leftarrow S$               | Warunkowe kopiowanie (zamiast skoku).               |

### Stos i zarządzanie procedurami

|                        |                                                        |                                                       |
|------------------------|--------------------------------------------------------|-------------------------------------------------------|
| <code>push S</code>    | $\%rsp - = 8$<br>$M[\%rsp] \leftarrow S$               | Odkłada na stos (zmniejsza <code>%rsp</code> ).       |
| <code>pop D</code>     | $D \leftarrow M[\%rsp]$<br>$\%rsp + = 8$               | Zdejmuje ze stosu do D (zwiększa <code>%rsp</code> ). |
| <code>call dest</code> | $\text{push } \%rip$<br>$\%rip \leftarrow \text{dest}$ | Skok do funkcji (zapisuje adres powrotu na stosie).   |
| <code>ret</code>       | $\text{pop } \%rip$                                    | Zdejmuje adres powrotu ze stosu i skacze pod niego.   |
| <code>leave</code>     | $\%rsp \leftarrow \%rbp$<br>$\text{pop } \%rbp$        | Sprząta ramkę stosu (odwrotność prologu).             |

Operacje wektorowe

Rozszerzenie AVX. Przedrostek `v` (nie niszczy źródeł).

|                                    |                            |                                                                                                                   |
|------------------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>vmovaps / ups S, D</code>    | $D \leftarrow S$           | Kopiuje wektor. <code>a</code> (aligned - szybkie, adres podzielny przez wyrównanie), <code>u</code> (unaligned). |
| <code>vaddps / pd S1, S2, D</code> | $D \leftarrow S2 + S1$     | Dodawanie wielokrotne (packed). <code>ps</code> (single-float), <code>pd</code> (double).                         |
| <code>vmulps / pd S1, S2, D</code> | $D \leftarrow S2 * S1$     | Mnożenie wektorowe.                                                                                               |
| <code>vfmadd231ps S1, S2, D</code> | $D \leftarrow S2 * S1 + D$ | <b>Fused Multiply-Add.</b> Mnoży i dodaje do <code>D</code> w jednym kroku.                                       |

Operacje zmiennoprzecinkowe

|                                   |                                |                                                                                                 |
|-----------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------|
| <code>movsd / movss S, D</code>   | $D \leftarrow S$               | Kopiuje skalar ( <code>double</code> / <code>float</code> ) między rejestrami XMM lub pamięcią. |
| <code>movapd / movaps S, D</code> | $D \leftarrow S$               | Kopiuje wyrównany wektor zmiennoprzecinkowy.                                                    |
| <code>addsd / subsd S, D</code>   | $D \leftarrow D \text{ op } S$ | Skalarne dodawanie / odejmowanie podwójnej precyzji.                                            |
| <code>xorpd / xorps S, D</code>   | $D \leftarrow D \hat{\ } S$    | Bitowy XOR na XMM. Jako <code>xorpd %xmm0, %xmm0</code> zeruje rejestr.                         |
| <code>ucomisd S1, S2</code>       | $S2 - S1$                      | Porównuje <code>double</code> i ustawia flagi <code>ZF, PF, CF</code> w <code>%rflags</code> .  |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `d` (32-bit), `q` (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).